Lab - Container Instances - Accessing Secrets within a Container
To Begin with the Lab

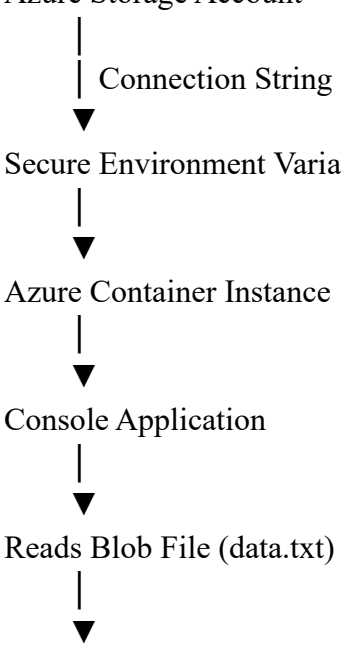
Summary
This lab demonstrates how to securely pass sensitive information (such as Azure Storage connection strings) to an Azure Container Instance (ACI) using **Secure Environment Variables**. The container reads the secret at runtime, accesses an Azure Storage Blob, and displays its contents without storing secrets inside the application code or Docker image.

Understanding
Applications often require sensitive information such as storage account keys, database passwords, API keys, or connection strings. Hardcoding these secrets into source code or Docker images is insecure because anyone with access to the image can retrieve them.
Azure Container Instances provides secure methods for supplying secrets to containers at runtime:

- **Secure Environment Variables** – Secret values are securely injected into the container as environment variables.
- **Secret Volume Mounts** – Secrets are mounted as files inside the container.

In this lab, the application retrieves the Azure Storage connection string from a secure environment variable. After receiving the connection string, it connects to Azure Blob Storage, reads the contents of a text file, and prints the data to the container logs.
Because this is a **console application**, it runs once, completes its task, and exits. Therefore, the container's **Restart Policy** is set to **Never**.

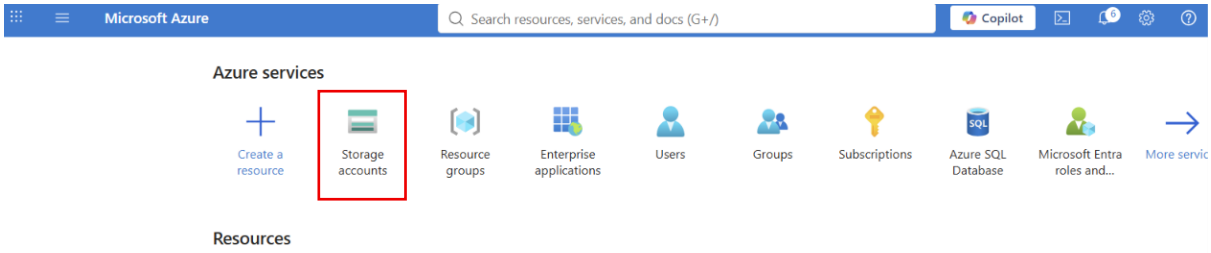
Example
Application Flow:



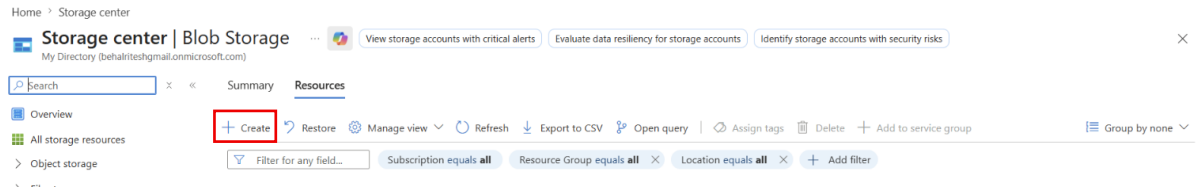
Lab Steps

Step 1: Create an Azure Storage Account

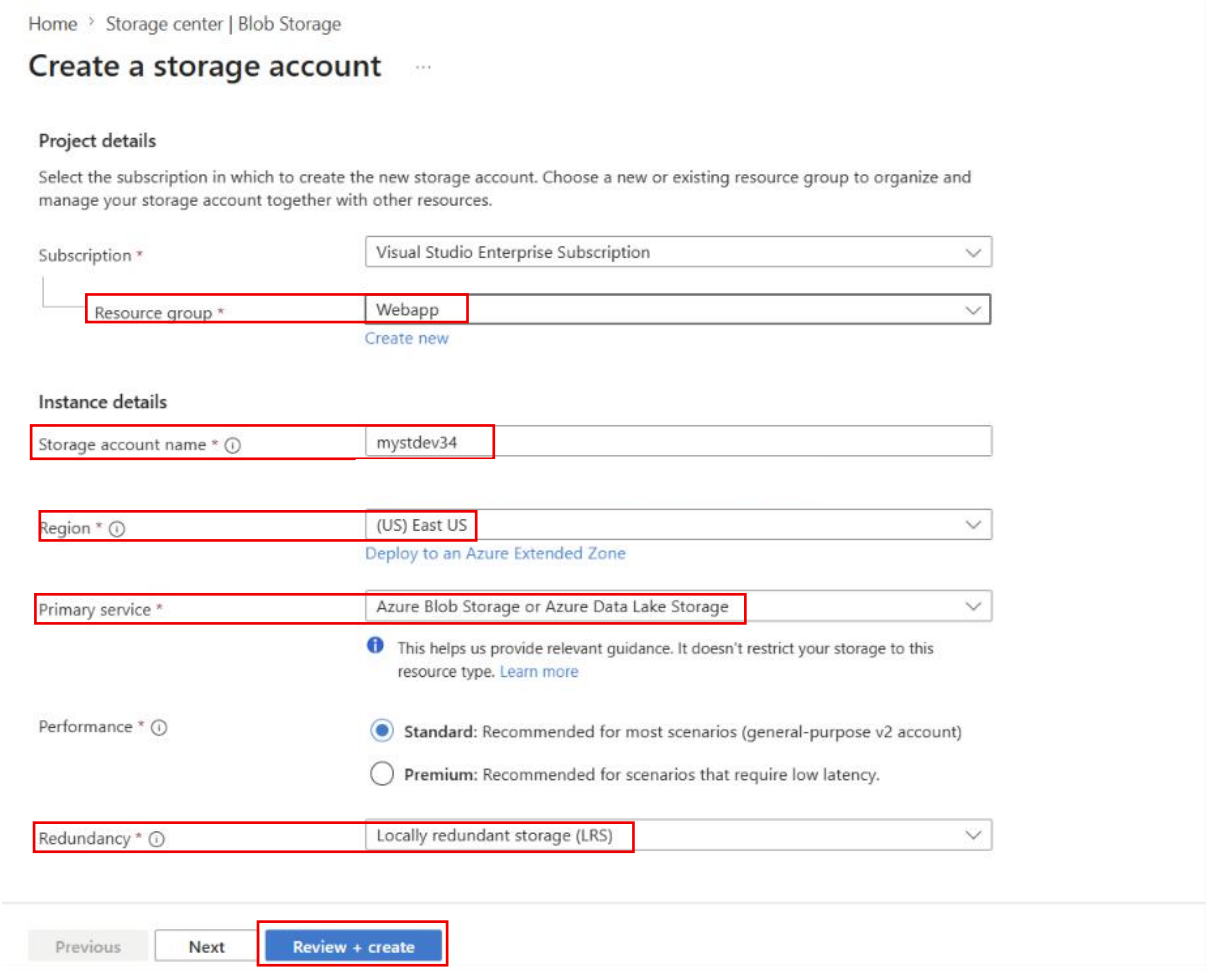
- Go to the **Storage accounts**.



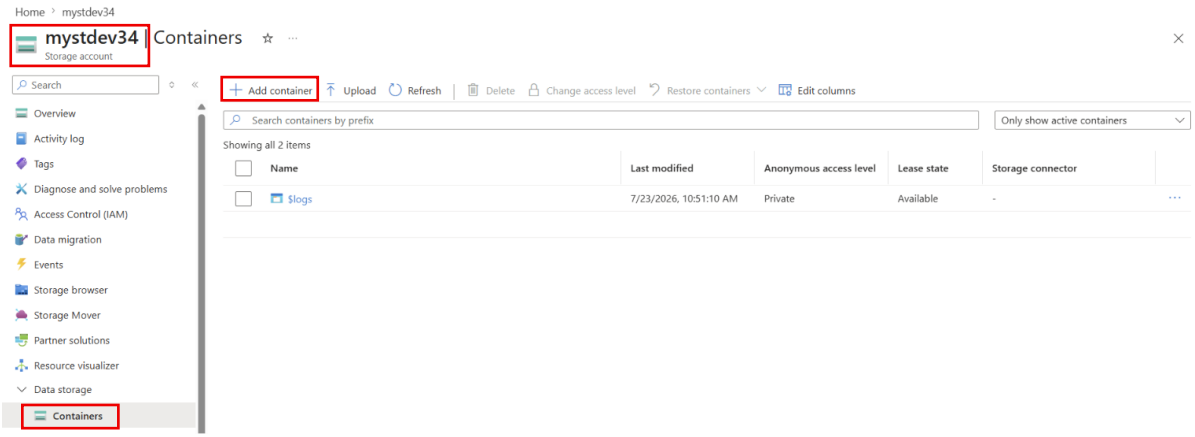
- Create an **Azure Storage Account**.



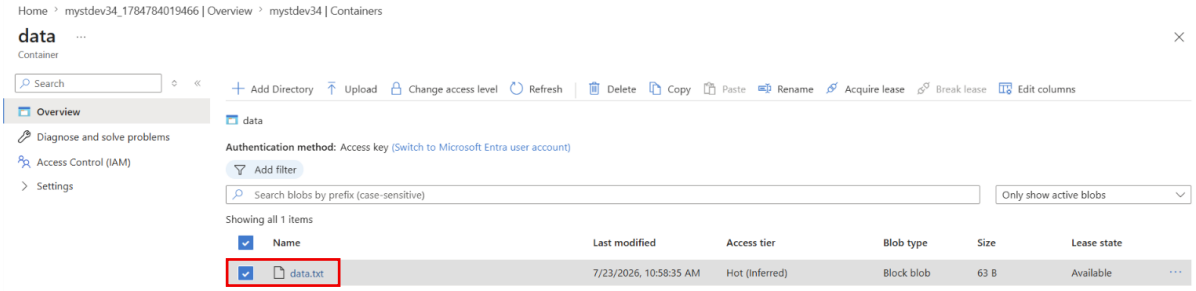
- **Subscription**.
- **Resource Group** (create a new one if needed).
- **Storage account name** (must be globally unique).
- **Region** (US) East US.
- **Performance:** Standard.
- **Redundancy:** Locally redundant storage (or as required).
- Click **Review + Create.**
- Click **Create.**



- Go to storage account just created.
- Navigate to Container.
- Create a **Data Container**.

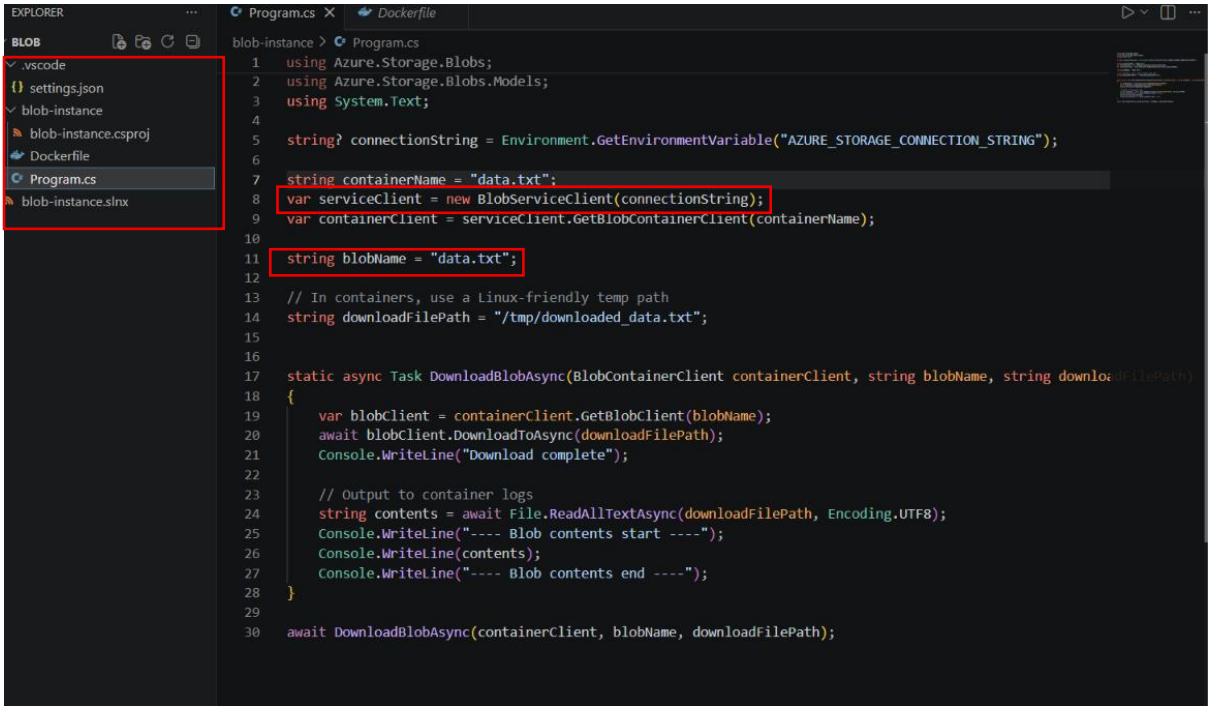


- Upload a text file (**data.txt**) into the container.
- Add sample text inside the file.



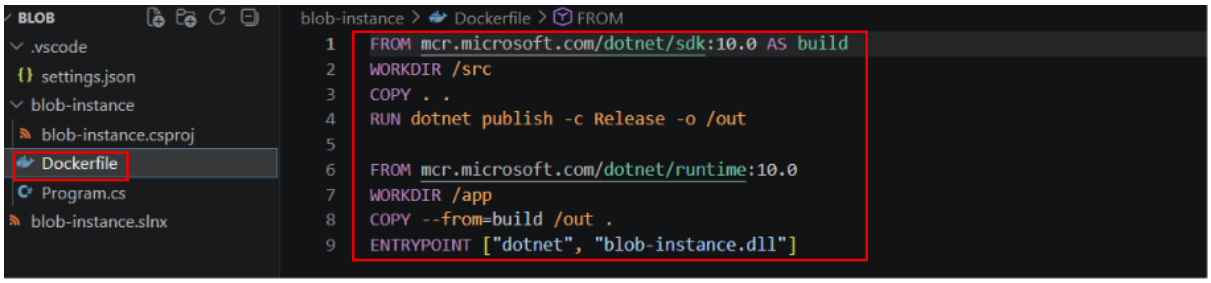
Step 2: Create the Console Application

- Develop a .NET console application.
- Read the storage connection string from an environment variable.
- Connect to Azure Blob Storage.
- Read the contents of the blob.



Step 3: Create the Dockerfile

- Create a multi-stage Dockerfile.
- Build the application.
- Publish the application.
- Configure the container to run the application DLL.

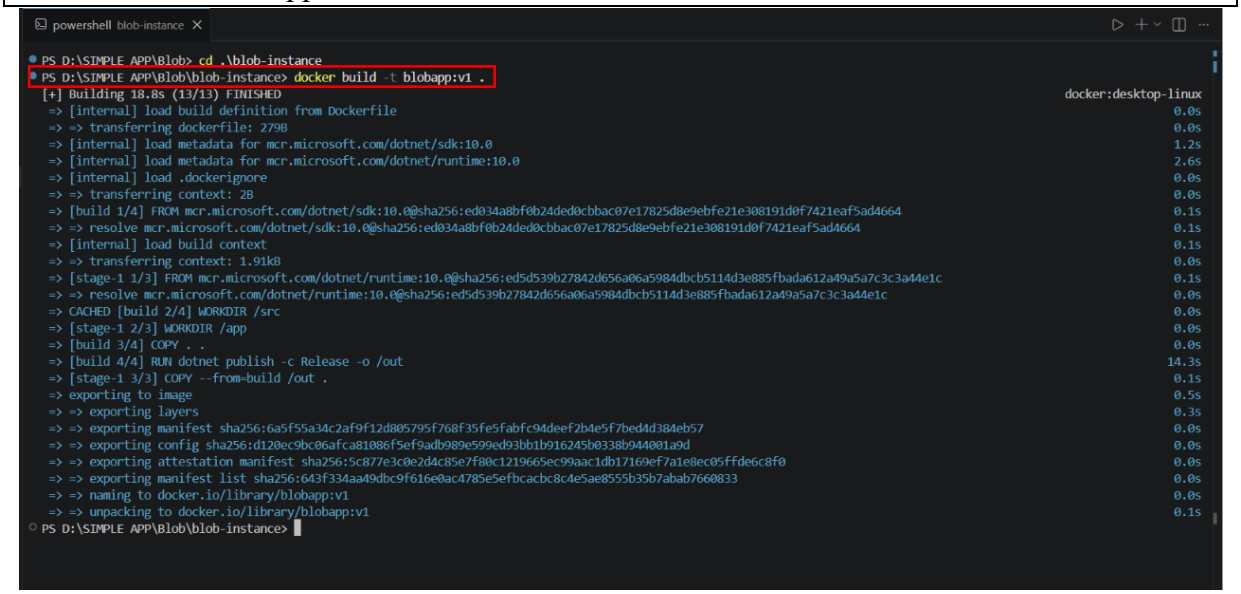


Step 4: Build the Docker Image

- Open the terminal.
- Navigate to the project directory.

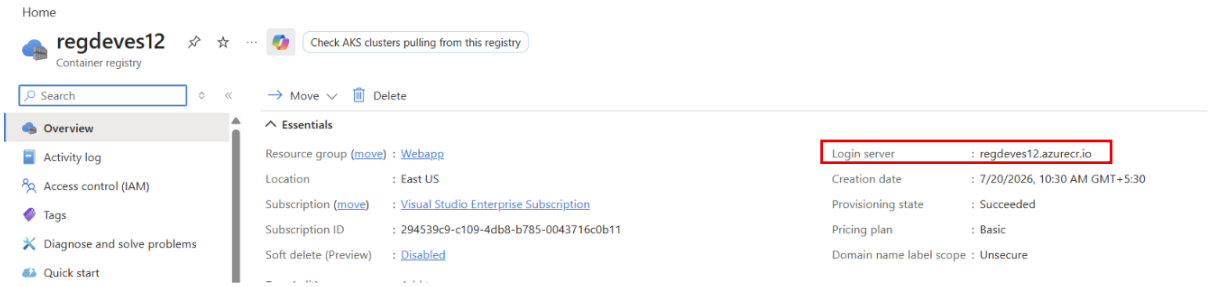
- Build the Docker image.

```
docker build -t blobapp:v1 .
```



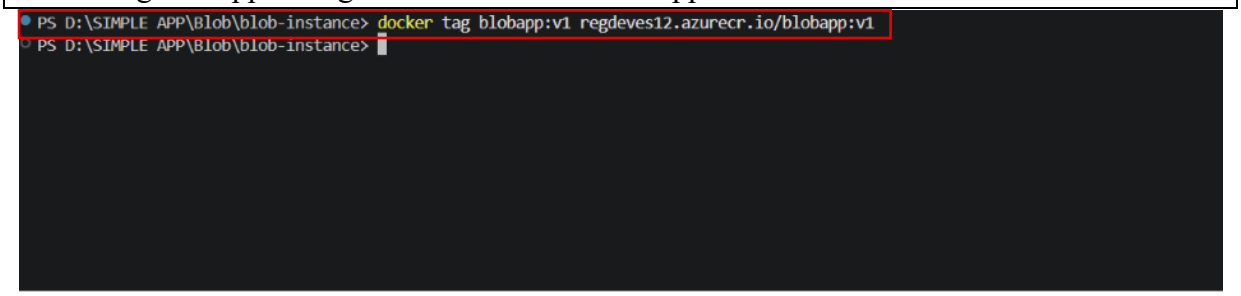
Step 5: Tag the Docker Image

- Go to the **container rejistry**.
- Copy the **Login server**.



- Tag the local Docker image.
- Associate it with the Azure Container Registry repository.
- Paste inside the command.

```
docker tag blobapp:v1 regdeves12.azurecr.io/blobapp:v1
```



Step 6: Push the Image to Azure Container Registry

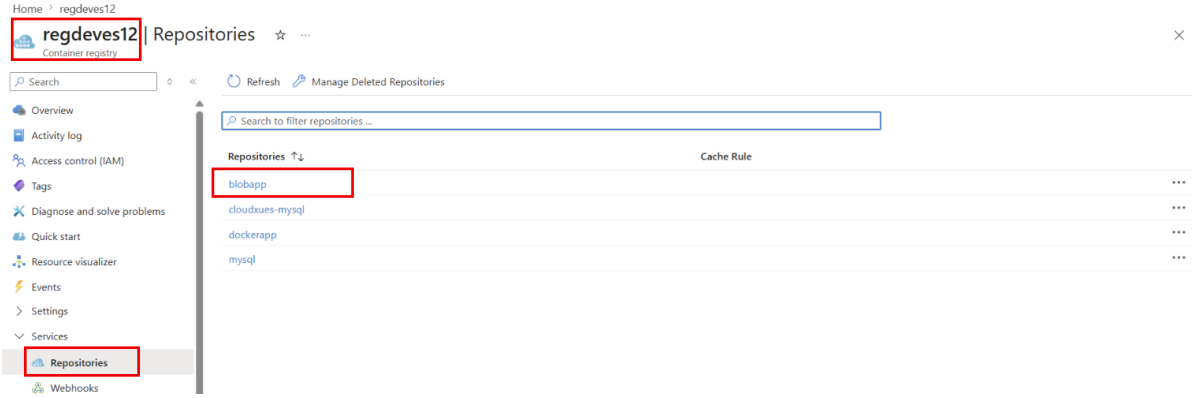
- Log in to Azure Container Registry.
- Push the image to the registry.
- Paste the login server inside this command.

```
docker push regdeves12.azurecr.io/blobapp:v1
```

```
PS D:\SIMPLE APP\Blob\blob-instance> docker tag blobapp:v1 regdeves12.azurecr.io/blobapp:v1
PS D:\SIMPLE APP\Blob\blob-instance> docker push regdeves12.azurecr.io/blobapp:v1
The push refers to repository [regdeves12.azurecr.io/blobapp]
ca2678b20700: Mounted from dockerapp
edf564afb68f: Mounted from dockerapp
7090f4cc64df: Mounted from dockerapp
8e782b788f5d: Mounted from dockerapp
65831f9a448c: Pushed
0287f08f0ac0: Mounted from dockerapp
a7e36c4b28c5: Pushed
5e727aa3601f: Pushed
v1: digest: sha256:643f334aa49dbc9f616e0ac4785e5efbcacbc8c4e5ae8555b35b7abab7660833 size: 856
PS D:\SIMPLE APP\Blob\blob-instance>
```

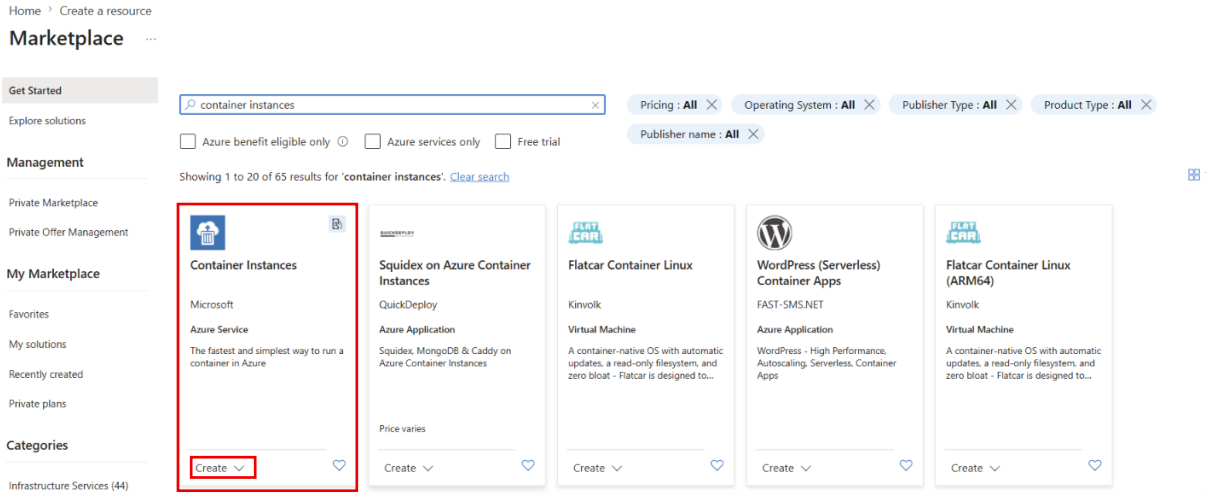
Step 7: Verify the Image

- Open Azure Portal.
- Navigate to **Azure Container Registry**.
- Open **Repositories**.
- Confirm that the **blobapp:v1** image exists.

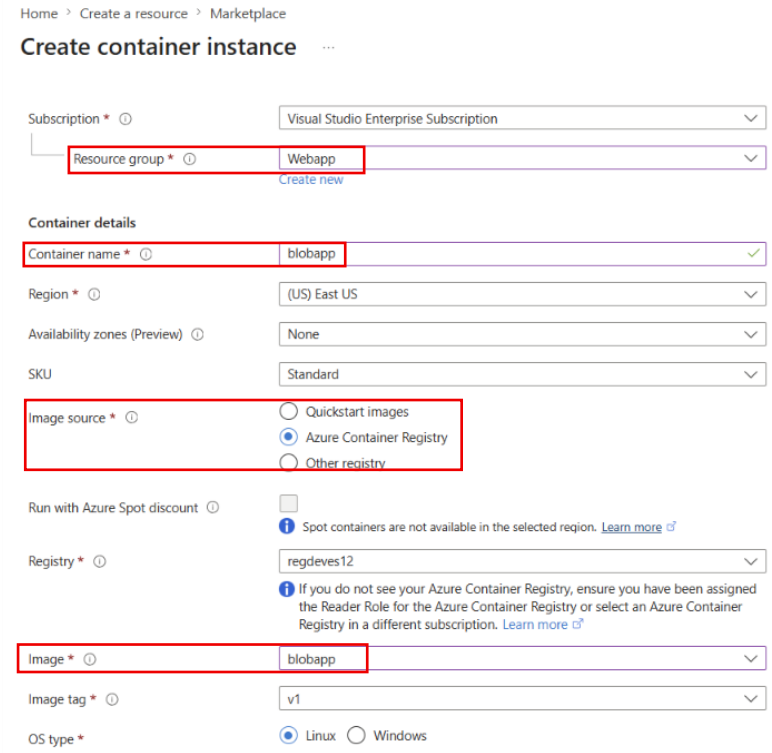


Step 8: Create an Azure Container Instance

- Open Azure Portal.
- Create a new **Azure Container Instance**.

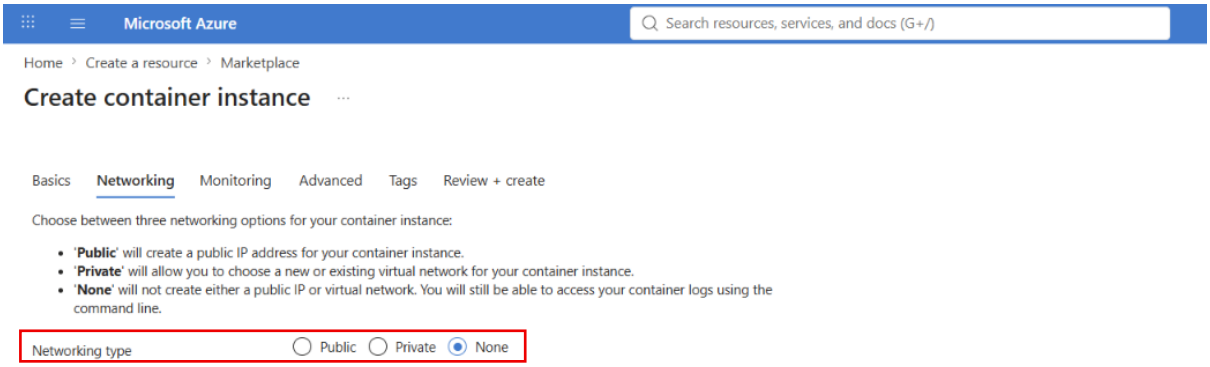


- Select the **Resource Group**.
- Enter the any **container name**.
- Select the **deployment region**.
- Choose the **Azure Container Registry**.
- Select the **blobapp:v1** image.



Step 9: Configure Networking

- Open the **Networking** section.
- Select **None** because public access is not required.

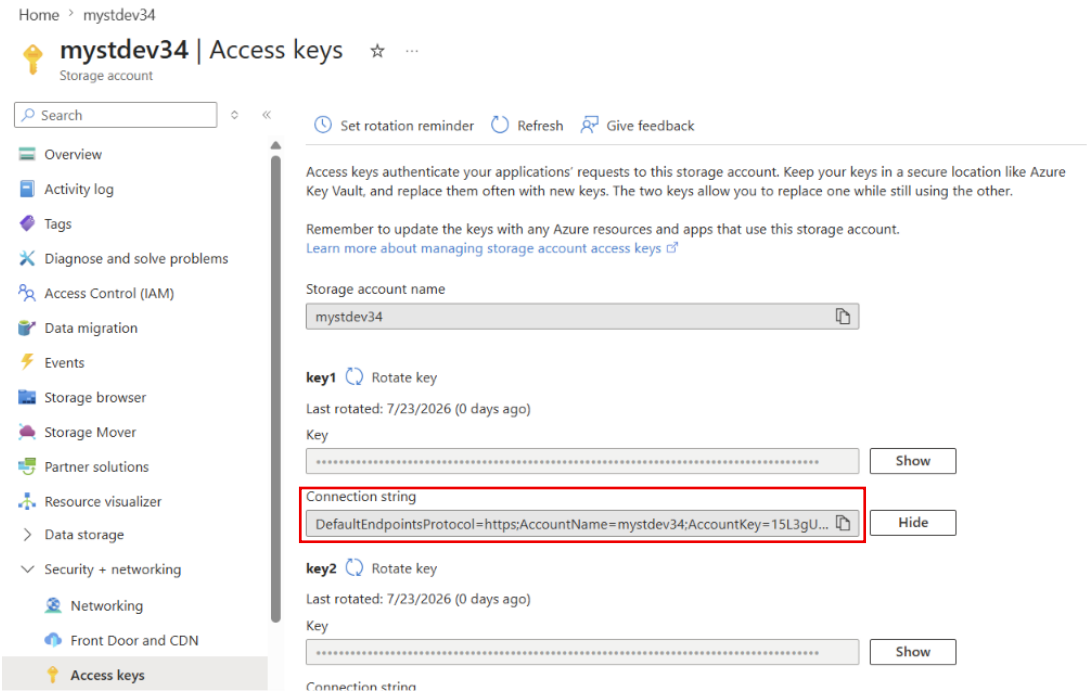


Step 10: Configure Secure Environment Variable

- Open the **Advanced** section.
- Add a new Environment Variable.
- Enter the variable name.
- Mark it as **Secure (Yes)**.
- Copy the Azure Storage connection string from the Storage Account.
- Paste it as the secure value.

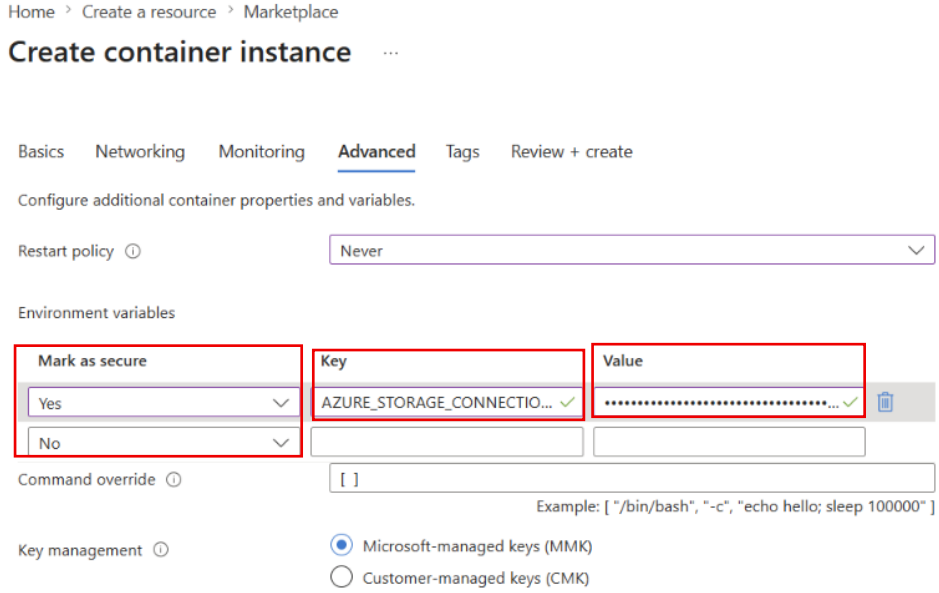
Variable Name:**AZURE_STORAGE_CONNECTION_STRING**

- Go to the Storage account.
- Navigate to Access keys.
- Copy the **Connection string**.



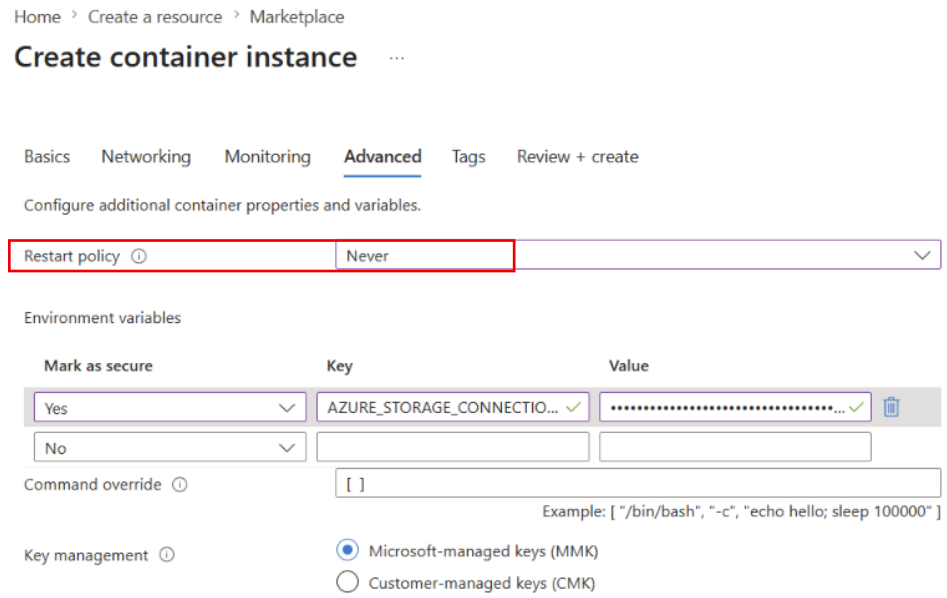
- Paste it as the secure value.

Value: <Storage Connection String>



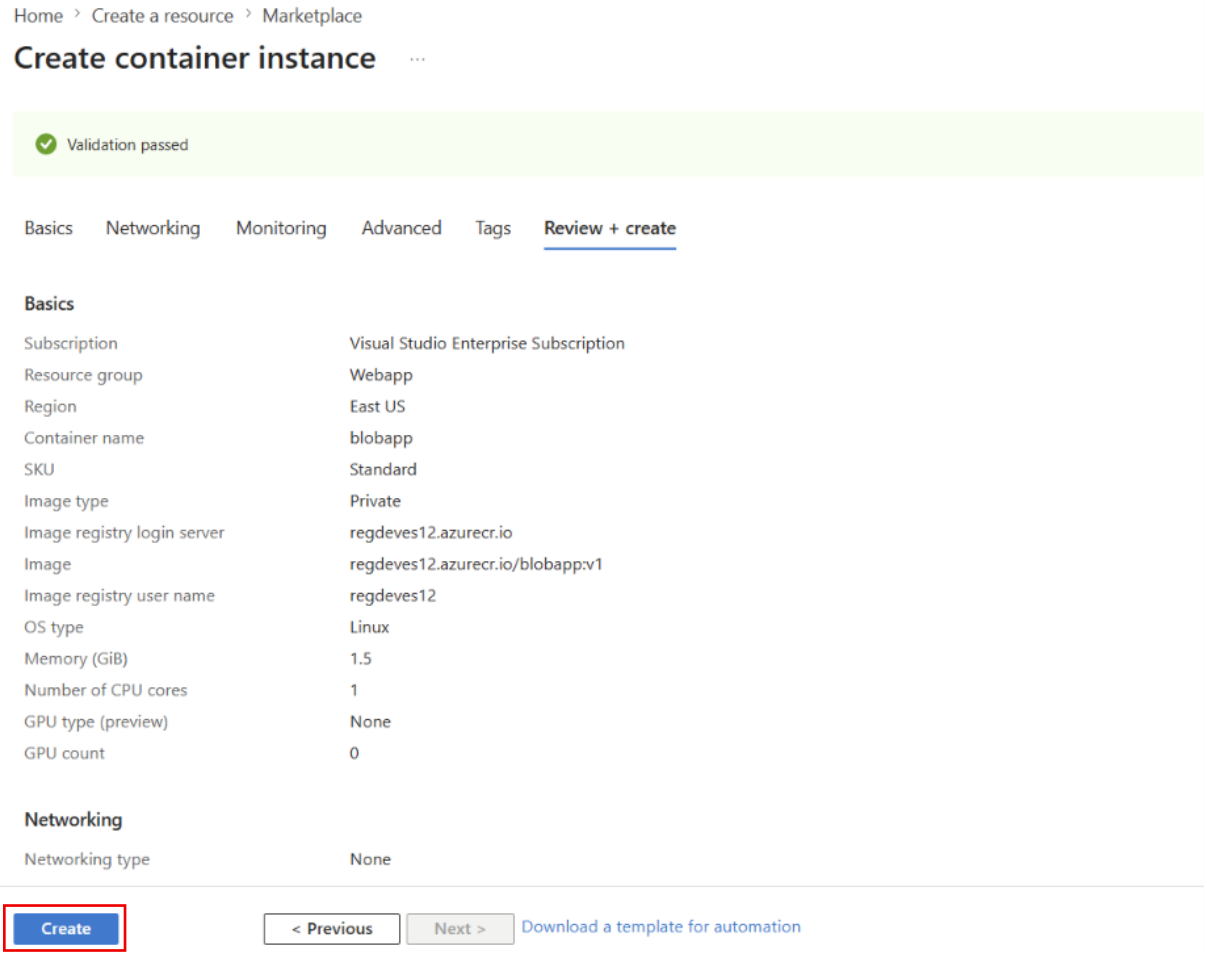
Step 11: Configure Restart Policy

- Set **Restart Policy** to **Never**.
- Since the application is a console application, it only needs to execute once.



Step 12: Deploy the Container

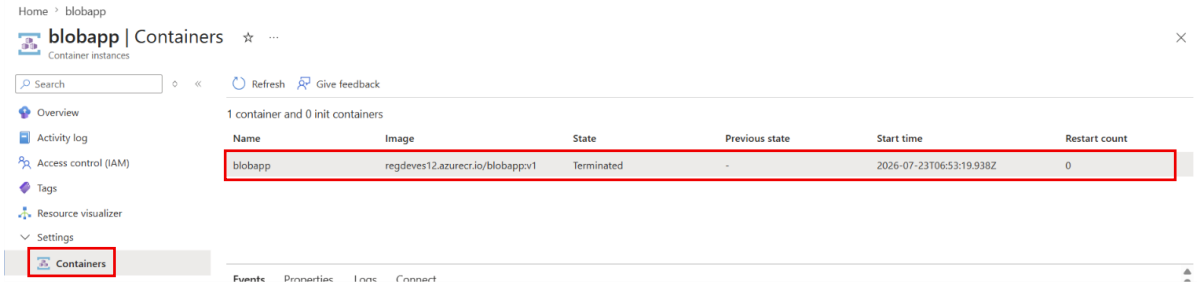
- Click **Review + Create**.
- Validate the configuration.
- Create the Azure Container Instance.



Step 13: Verify Container Status

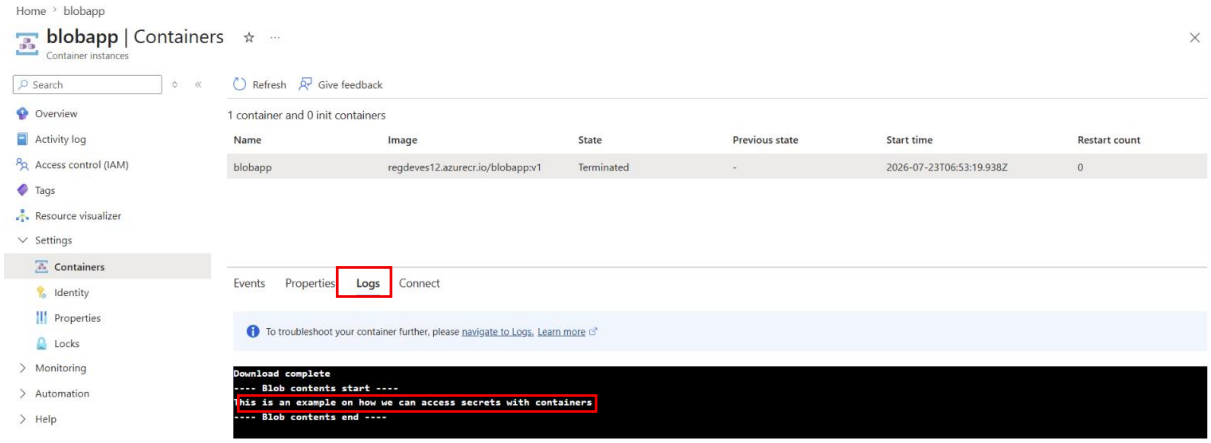
- Open the deployed Container Instance.
- Navigate to **Containers**.

- Verify that the container has completed execution.
- The status should be **Terminated**.



Step 14: View Container Logs

- Open the **Logs** section.
- Verify that the contents of **data.txt** are displayed.
- Confirm that the application successfully connected to Azure Storage.



Step 15: Understand Azure File Share Mounts

- Azure Container Instances also support Azure File Share mounting.
- Mount Azure File Shares as persistent storage.
- Data stored in Azure File Shares remains available even after the container stops.
- Use Azure File Shares whenever persistent storage is required.

Important Commands

Build Docker Image

docker build -t blobapp:v1 .

Tag Docker Image

docker tag blobapp:v1 <acr-name>.azurecr.io/blobapp:v1

Push Docker Image

docker push <acr-name>.azurecr.io/blobapp:v1